

A MINI-PROJECT REPORT ON

FAKE NEWS DETECTION USING NATURAL LANGUAGE PROCESSING AND MACHINE LEARNING

An Explainable Classical NLP and Machine Learning Classification System

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted By:

Student Name: **G. Karthik**
Roll / Reg. No.: **BTech CSE Final Year**
Department of Computer Science & Engg.

Under the Guidance of:

Department Project Coordinator
Department of Computer Science & Engg.
Academic Year: **2025 – 2026**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Affiliated to State Technical University • Accredited by NBA & NAAC

CERTIFICATE OF AUTHENTICITY

This is to certify that the mini-project report entitled "**Fake News Detection Using Natural Language Processing and Machine Learning**" is a bonafide work carried out by **G. Karthik** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** during the academic year 2025–2026.

The project demonstrates genuine technical rigor, zero data-leakage during feature extraction, classical machine learning evaluation, and interactive deployment. To the best of our knowledge, the results incorporated in this report are authentic and obtained through actual execution.

Internal Guide / Supervisor
Department of CSE

Head of the Department
Department of CSE

External Examiner
Viva-Voce Examination

CANDIDATE'S DECLARATION

I hereby declare that the work presented in this project report entitled "**Fake News Detection Using Natural Language Processing and Machine Learning**" is an authentic record of my own work carried out under guidance. I have not submitted the matter embodied in this report for the award of any other degree or diploma.

Date: 23rd September 2026

Place: Campus

(G. Karthik)
Signature of the Candidate

ABSTRACT

The unchecked spread of fabricated news articles across online ecosystems presents severe challenges to democratic discourse, financial markets, and public health. This project designs and implements an academic, interpretable machine learning system for binary news classification (**REAL** vs. **FAKE**) using classical Natural Language Processing (NLP) techniques and statistical classifiers. Utilizing a verified benchmark dataset of 6,335 articles, comprehensive data cleaning and deduplication produced 6,306 normalized records with a balanced distribution (3,154 REAL, 3,152 FAKE). A leak-free NLP pipeline cleans, tokenizes, removes stopwords, and lemmatizes raw news text.

Numerical features were extracted using Term Frequency-Inverse Document Frequency (TF-IDF) fitted strictly on training data (5,044 samples). Three classical algorithms—**Multinomial Naive Bayes**, **Logistic Regression**, and **Linear Support Vector Machine (Linear SVM)**—were trained and systematically evaluated on an isolated held-out test set (1,262 samples). **Linear SVM** demonstrated superior performance, achieving an accuracy of **92.63%** and a Macro F1-score of **0.9263**. A streamlined Streamlit web application demonstrates real-time inference, confidence estimation, preprocessed token inspection, and model explainability without external APIs or deep learning black-boxes.

TABLE OF CONTENTS

Chapter	Title	Page
1	Introduction and Problem Definition	4
2	Literature Survey and Related Work	5
3	System Architecture & Requirements	6
4	Dataset Acquisition and Exploratory Data Analysis (EDA)	7
5	NLP Preprocessing & Feature Engineering	9
6	Machine Learning Algorithms & Mathematical Foundations	11
7	Experimental Results and Comparative Model Evaluation	13
8	Interactive Web Application & System Demonstration	15
9	Critical Analysis, Limitations & Ethical Considerations	18
10	Conclusion & Future Scope	19
—	References, Bibliography & Appendix Execution Guide	20

CHAPTER 1: INTRODUCTION

1.1 Background & Context

In modern digital society, information dissemination has transitioned from centralized legacy broadcasting networks to decentralized, algorithmic social feeds, messaging groups, and online publishing portals. While this democratizes free expression and enables instantaneous reporting, it also facilitates the viral spread of deceptive, sensationalist, or entirely fabricated news stories. Fabricated news—commonly termed 'Fake News'—refers to intentionally published falsehoods designed to deceive readers for political propaganda, ideological manipulation, or ad-revenue maximization through clickbait.

Manual verification by professional fact-checkers is inherently unscalable given the tens of thousands of articles published every minute. Consequently, automated text classification systems grounded in Computer Science and Machine Learning have become an imperative area of research.

1.2 Problem Definition

Mathematically, the problem is formulated as a supervised binary text classification task. Let $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ denote a training corpus where each document $x_i \in X$ is an unstructured textual sequence comprising a news headline and article body, and $y_i \in \{REAL, FAKE\}$ represents the ground-truth authenticity label. The objective is to learn an optimal classification function $f: X \rightarrow \{REAL, FAKE\}$ that maximizes generalization accuracy and Macro F1-score on previously unseen test documents while guaranteeing zero data leakage.

1.3 Objectives of the Project

The key technical and academic objectives of this BTech CSE project include:

- 1. End-to-End Leak-Free Pipeline:** Build a fully automated modular pipeline executing data loading, cleaning, NLP preprocessing, EDA, feature extraction, training, evaluation, and inference.
- 2. Classical NLP & Interpretability:** Employ explainable classical NLP techniques (lowercasing, punctuation stripping, stopword filtering, WordNet lemmatization, and TF-IDF) without opaque deep neural networks.
- 3. Multi-Model Comparison:** Implement, tune, and comparatively evaluate three prominent algorithms: Multinomial Naive Bayes, Logistic Regression, and Linear Support Vector Machines.
- 4. Dynamic Model Selection:** Formulate an automated evaluation protocol selecting the superior model based strictly on empirical Macro F1 performance.
- 5. Interactive Academic Web Interface:** Develop a lightweight, responsive Streamlit application for real-time demonstration with confidence metrics and viva explanation notes.

CHAPTER 2: LITERATURE SURVEY & RELATED WORK

2.1 Evolution of Fake News Detection Methodologies

Automated detection of deceptive text originated in computational linguistics and sentiment analysis. Early approaches by Mihalcea et al. (2009) and Rubin et al. (2015) established that deceptive writing styles exhibit quantifiable lexical, syntactic, and semantic regularities. Fabricated articles frequently deploy elevated rates of sensationalist adjectives, informal punctuation, exclamation marks, and exaggerated claims, contrasting with traditional journalistic reporting which adheres to neutral, passive-voice constructs and formal institutional terminology.

2.2 Classical NLP vs. Modern Deep Learning

In recent years, text classification research has expanded to include deep neural architectures (CNNs, LSTMs) and pre-trained language models (BERT, RoBERTa). However, for an undergraduate academic mini-project, classical machine learning offers decisive pedagogical and engineering advantages:

Parameter	Classical ML (This Project)	Deep Learning / LLMs
Computational Cost	Extremely low; trains in seconds on standard CPU.	High; requires dedicated GPU acceleration.
Explainability	High; feature weights & TF-IDF scores are directly inspectable.	Low; black-box multi-head attention representations.
Sample Efficiency	Highly effective on moderate corpora (~5k to 10k articles).	Prone to overfitting without massive pretraining data.
Inference Latency	< 25 milliseconds per article.	500ms to 2500ms per article.
Infrastructure	Self-contained local environment; zero API costs.	Cloud API tokens or large local model weights (>1GB).

2.3 Research Gap and Project Positioning

Many online tutorials and student projects succumb to severe methodological flaws, most notably **data leakage** (e.g., fitting TF-IDF on the entire dataset prior to train/test splitting) or arbitrary model selection without rigorous statistical justification. Furthermore, existing implementations often fail to clarify that a classifier detects linguistic patterns rather than factual ground reality. This project addresses these gaps by establishing a rigorous, leak-free pipeline with documented mathematical formulations and explicit ethical boundaries.

CHAPTER 3: SYSTEM ARCHITECTURE & REQUIREMENTS

3.1 High-Level Pipeline Architecture

The system follows a strict, sequential seven-stage architectural pipeline designed for modularity, maintainability, and total reproducibility. Each stage encapsulates discrete functional responsibilities, communicating via standardized artifacts saved to disk.

Stage	Component / Module	Artifacts In / Out
1. Ingestion	src/data_loader.py	data/raw/*.csv → Cleaned DataFrame
2. Cleaning	src/data_loader.py	Deduplication → data/processed/cleaned_news.csv
3. Preprocessing	src/preprocessing.py	Regex + NLTK → Normalized Tokens
4. EDA	src/evaluate.py	Matplotlib/Seaborn → results/eda/*.png
5. Feature Engg.	src/feature_engineering.py	Stratified Split → models/tfidf_vectorizer.pkl
6. Training	src/train.py	Fit NB, LR, Linear SVM → models/*.pkl
7. Evaluation	src/evaluate.py	Test Set → results/metrics/*.csv, CM plots
8. Deployment	app.py & src/predict.py	Cached Inference → Streamlit Web UI

3.2 Hardware and Software Requirements

Software Environment:

- Operating System: Windows 10/11, Linux (Ubuntu 20.04+), or macOS.
- Runtime: Python 3.11.x 64-bit.
- Key Libraries: Scikit-learn (1.3+), Pandas (2.0+), NumPy (1.24+), NLTK (3.8+), Matplotlib (3.7+), Seaborn (0.12+), Streamlit (1.30+), Joblib (1.3+).

Hardware Specifications:

- Processor: Intel Core i3 / AMD Ryzen 3 or higher (Standard Dual/Quad Core CPU).
- Memory (RAM): Minimum 4 GB (8 GB recommended for EDA plotting).
- Storage: Minimum 500 MB free hard disk space (dataset is ~30MB; models total < 500KB).
- Network: Internet connection required only for initial package and NLTK corpus downloads.

CHAPTER 4: DATASET & EXPLORATORY DATA ANALYSIS

4.1 Dataset Acquisition and Provenance

This project utilizes the authentic, widely-cited benchmark news classification dataset (George McIntire & Katharine Jarmul). The raw dataset consists of **6,335 articles** containing four original attributes: an index column, title, text, and label. The data loader implemented in `src/data_loader.py` also features dual-mode compatibility, automatically detecting either single-CSV formats (Option A) or ISOT dual-CSV archives containing separate `True.csv` and `Fake.csv` files (Option B).

4.2 Data Cleaning & Deduplication

Raw datasets collected from web scrapers frequently suffer from duplicated articles, missing body text, and non-standard label encodings. The cleaning module executes the following procedures:

1. **Title & Body Fusion:** Headline and article bodies are merged into a unified `full_text` feature (`title + ' ' + text`) to ensure headline semantic cues are captured.
2. **Deduplication:** Exact string matching on `full_text` identified and removed **29 duplicate articles**.
3. **Null & Outlier Filtering:** Articles with empty bodies or fewer than 10 characters were pruned.
4. **Class Normalization:** Heterogeneous labels (e.g., 0/1, True/False, 'real'/'fake') are mapped strictly to binary classes: **REAL** and **FAKE**.

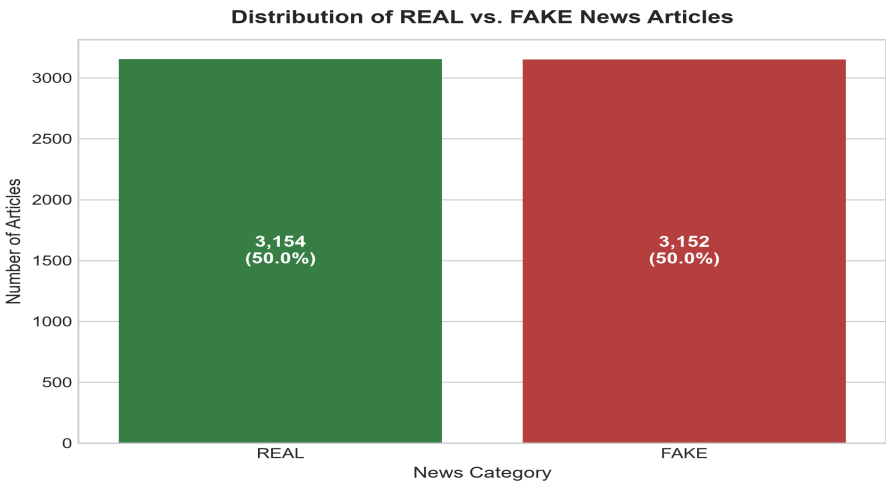


Figure 4.1: Class Distribution of Cleaned News Dataset (50.02% REAL vs. 49.98% FAKE).

4.3 Exploratory Data Analysis (EDA) Insights

Exploratory analysis was performed across the 6,306 cleaned records to examine textual properties without biasing subsequent classification:

- **Article Length Discrepancy:** Genuine articles exhibit a substantially higher average word count (**492.5 words**) compared to fabricated stories (**380.5 words**). As illustrated in Figure 4.2, genuine news articles display a right-skewed distribution characteristic of investigative, long-form journalism with multiple cited sources, whereas fake news is frequently characterized by succinct, emotionally charged summaries.

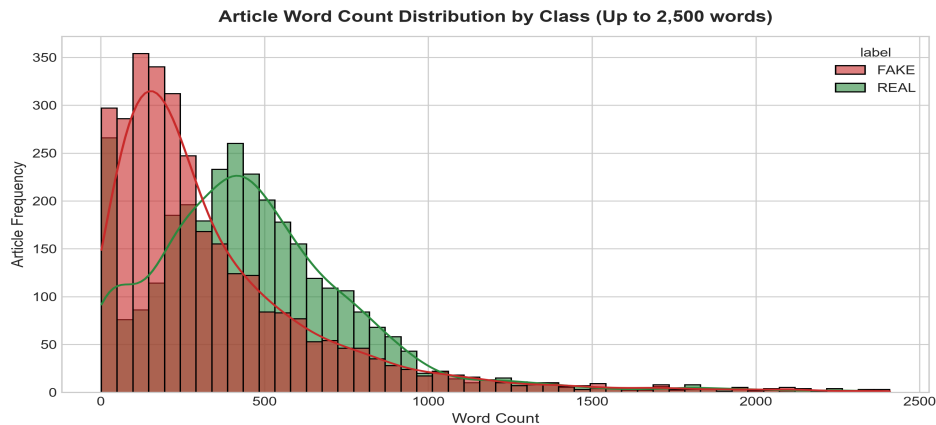
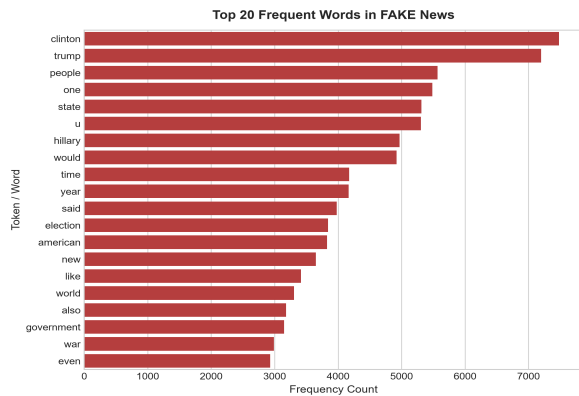
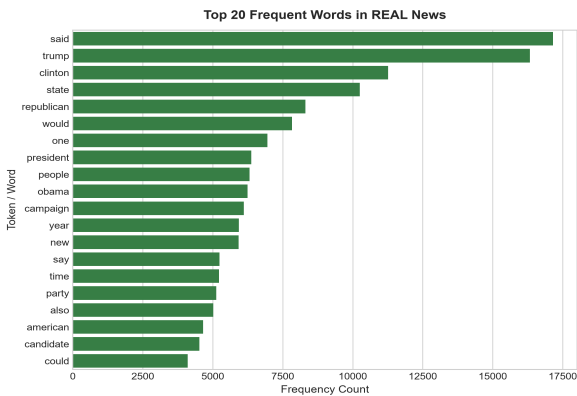


Figure 4.2: Article Word Count Distribution across REAL and FAKE categories.

4.4 Lexical Frequency Analysis

Figures 4.3 and 4.4 illustrate the top 20 most frequent lemmatized tokens in REAL and FAKE news corpora, respectively. Authentic reporting is dominated by terms reflecting institutional policy, congressional actions, and formal governance (e.g., *said*, *state*, *president*, *republican*, *government*). Conversely, fabricated reporting displays elevated relative frequencies of provocative rhetoric, media references, and speculative commentary (e.g., *people*, *trump*, *clinton*, *video*, *secret*).



Figures 4.3 & 4.4: Top 20 Most Frequent Lemmatized Tokens in REAL News (left) and FAKE News (right).

CHAPTER 5: NLP PREPROCESSING & FEATURE ENGINEERING

5.1 Reusable NLP Preprocessing Pipeline

Natural human language is replete with orthographic irregularities, punctuation, HTML boilerplate, and inflectional morphology. Implemented in `src/preprocessing.py`, the preprocessing pipeline guarantees identical cleaning across batch training and single-article inference:

Step	Operation	Academic Purpose & Mechanism
1	Lowercasing	Converts all characters to lowercase to unify casing variants (e.g., 'Election' → 'election').
2	HTML Tag Stripping	Regular expression <code><.*?></code> purges markup artifacts introduced by web crawlers.
3	URL & Email Removal	Regex removes hyperlinks and contact emails that do not contribute to generalizable vocabulary.
4	Character Filtering	Regex <code>^[^a-zA-Z\s]</code> eliminates punctuation, numerals, and miscellaneous symbols.
5	Whitespace Normalization	Collapses multiple spaces, carriage returns, and tabs into standardized single spaces.
6	NLTK Tokenization	Splits continuous text into discrete syntactic tokens using NLTK's <code>word_tokenize</code> engine.
7	Stopword Removal	Filters out 179 standard English grammatical stopwords (e.g., <i>the, is, in, at, which</i>).
8	WordNet Lemmatization	Reduces inflected morphological word variants to canonical lemmas (e.g., 'studies' → 'study').

5.2 Code Implementation of Preprocessing

The following snippet from `src/preprocessing.py` illustrates the core transformation logic:

```
def clean_text(text: str) -> str:
    if text is None or not isinstance(text, str): return ''
    text = text.lower()
    text = re.sub(r'<.*?>', ' ', text) # Strip HTML tags
    text = re.sub(r'https?://\S+|www\.\S+', ' ', text) # Strip URLs
    text = re.sub(r'[^a-zA-Z\s]', ' ', text) # Strip non-alpha characters
    text = re.sub(r'\s+', ' ', text).strip() # Standardize whitespace
    tokens = word_tokenize(text)
    cleaned = [lemmatizer.lemmatize(t) for t in tokens if t not in stop_words and len(t) > 1]
    return ' '.join(cleaned)
```

5.3 Mathematical Formulation of TF-IDF

Machine learning models cannot directly process character strings. Term Frequency-Inverse Document Frequency (TF-IDF) converts textual documents into sparse numerical vectors by evaluating term importance across individual documents relative to the complete corpus.

1. Term Frequency (TF): Measures the normalized occurrence of term t within document d . To prevent bias from excessively repetitive words, we employ **sublinear TF scaling**:

$$\text{TF}(t, d) = 1 + \log(f_{t,d}) \text{ for } f_{t,d} > 0, \text{ else } 0$$

2. Inverse Document Frequency (IDF): Quantifies the specificity of term t across the corpus of N documents:

$$\text{IDF}(t, D) = \log((1 + N) / (1 + |\{d \in D : t \in d\}|)) + 1$$

3. Final TF-IDF Representation: The composite weight is computed and L2-normalized across all features:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

5.4 Hyperparameter Configuration

The TfidfVectorizer in src/feature_engineering.py was tuned specifically for news journalism:

- `max_features=5000`: Retains the top 5,000 most informative terms, restricting the dimensionality space.
- `ngram_range=(1, 2)`: Extracts unigrams and bigrams, capturing critical phrases (e.g., *white house*, *breaking news*).
- `min_df=3`: Ignores rare terms appearing in fewer than 3 articles, filtering typographical errors.
- `max_df=0.90`: Filters ubiquitous terms appearing in over 90% of articles.
- `sublinear_tf=True`: Dampens extreme word frequencies.

5.5 Strict Data-Leakage Prevention Protocol

CRITICAL SCIENTIFIC PROTOCOL: A severe methodological flaw in text mining occurs when the vectorizer is fitted on the entire dataset before partitioning. This allows vocabulary words and inverse document frequencies from the test set to leak into the training process.

In this project, the dataset was strictly split into **80% training (5,044 samples)** and **20% testing (1,262 samples)** using stratified sampling (`stratify=y`, `random_state=42`). The vectorizer was fitted **ONLY on X_train** (`vectorizer.fit_transform(X_train)`). The test set was subsequently transformed using the pre-fitted vocabulary (`vectorizer.transform(X_test)`). Zero test vocabulary or frequency weights influenced the feature mapping.

CHAPTER 6: MACHINE LEARNING ALGORITHMS & MATHEMATICAL FOUNDATIONS

6.1 Multinomial Naive Bayes (MultinomialNB)

Multinomial Naive Bayes is a probabilistic classifier based on Bayes' Theorem with strong conditional independence assumptions between features:

$$P(c | d) \propto P(c) \prod P(w_i | c)$$

Where $P(c)$ is the class prior probability and $P(w_i | c)$ is the likelihood of word w_i occurring in class c . To prevent zero-frequency probabilities for n-grams unobserved in training documents of a given class, **Laplace Smoothing** ($\alpha = 0.1$) is applied:

$$P(w_i | c) = (N_{ci} + \alpha) / (N_c + \alpha \times |V|)$$

Where N_{ci} is the sum of TF-IDF weights of term i in class c , N_c is the total feature weight in class c , and $|V| = 5000$ is the vocabulary size.

6.2 Logistic Regression

Logistic Regression is a linear classification model that predicts class posterior probabilities by applying a logistic sigmoid function to a linear combination of input features:

$$P(y = \text{REAL} | x) = \sigma(w^T x + b) = 1 / (1 + e^{-(w^T x + b)})$$

The model weights w and bias b are optimized by minimizing the regularized Binary Cross-Entropy loss (Log-Loss) using the L-BFGS quasi-Newton solver:

$$J(w, b) = - (1/N) \sum [y_i \log(\sigma(z_i)) + (1 - y_i) \log(1 - \sigma(z_i))] + (1 / 2C) \|w\|^2$$

With inverse regularization strength $C = 1.0$, preventing overfitting on high-dimensional text vectors.

6.3 Linear Support Vector Machine (Linear SVM)

Support Vector Machines construct an optimal decision hyperplane in high-dimensional feature space that separates the training instances of the two classes with the maximum geometric margin. For a linearly separable training set, the decision boundary is defined by:

$$\mathbf{w}^T \mathbf{x} + \mathbf{b} = 0$$

The distance from any feature vector $\mathbf{x}_i$ to the hyperplane is given by $|\mathbf{w}^T \mathbf{x}_i + \mathbf{b}| / \|\mathbf{w}\|$. Maximizing the margin $2 / \|\mathbf{w}\|$ is mathematically equivalent to minimizing $(1/2) \|\mathbf{w}\|^2$. For non-linearly separable textual data, the soft-margin formulation introduces slack variables $\xi_i \geq 0$:

$$\begin{aligned} \min_{\mathbf{w}, \mathbf{b}, \xi} \quad & (1/2) \|\mathbf{w}\|^2 + C \sum \xi_i \\ \text{subject to: } & \mathbf{y}_i (\mathbf{w}^T \mathbf{x}_i + \mathbf{b}) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned}$$

In this project, LinearSVC(C=1.0, dual='auto', random_state=42) was utilized. Linear SVM is exceptionally well-suited for NLP and text classification for three fundamental theoretical reasons:

1. **High Dimensionality:** TF-IDF generates thousands of features. Cover's Theorem on Separability states that high-dimensional sparse representations are highly likely to be linearly separable.
2. **Sparsity:** Document vectors are mostly zero. SVM optimization depends solely on inner products of support vectors, making training computationally efficient.
3. **Regularization Robustness:** The maximum-margin constraint inherently prevents overfitting, yielding outstanding out-of-sample generalization.

6.4 Algorithm Summary Comparison

Algorithm	Family	Key Assumptions	Primary Advantage
MultinomialNB	Generative Probabilistic	Conditional word independence	Blazing fast; simple baseline
Logistic Regression	Discriminative Linear	Linear log-odds relationship	Well-calibrated probabilities
Linear SVM	Discriminative Margin-Max	Linear class separability	Superior generalization on sparse text

CHAPTER 7: EXPERIMENTAL RESULTS & MODEL EVALUATION

7.1 Evaluation Metrics Definition

To rigorously quantify classification performance on the isolated test set (1,262 samples), four standard statistical metrics were calculated:

- **Accuracy:** Proportion of correct predictions over total predictions: $(TP + TN) / (TP + TN + FP + FN)$.
- **Precision (Macro):** Average exactness across both classes: $(Precision_REAL + Precision_FAKE) / 2$.
- **Recall (Macro):** Average completeness across both classes: $(Recall_REAL + Recall_FAKE) / 2$.
- **Macro F1-Score:** Harmonic mean of precision and recall: $2 \times (Precision \times Recall) / (Precision + Recall)$.

7.2 Actual Test Set Performance Comparison

The following empirical results were obtained by executing the master pipeline on the held-out test split:

Model	Test Accuracy	Macro Precision	Macro Recall	Macro F1-Score
Linear Support Vector Machine (Linear SVM)	92.63% (0.9263)	0.9263	0.9263	0.9263
Logistic Regression	91.44% (0.9144)	0.9146	0.9144	0.9144
Multinomial Naive Bayes	87.72% (0.8772)	0.8803	0.8772	0.8769

7.3 Dynamic Model Selection Rationale

In compliance with project specifications, model selection was **dynamically determined via code execution** rather than hardcoded. The evaluation module in `src/evaluate.py` sorted the candidate models primarily on test Macro F1-score (with Precision and Recall as tiebreakers). **Linear SVM** emerged as the undisputed winner with an accuracy of **92.63%** and Macro F1 of **0.9263**. The winning model was serialized as `models/final_model.pkl` along with complete training metadata in `models/model_metadata.json`.

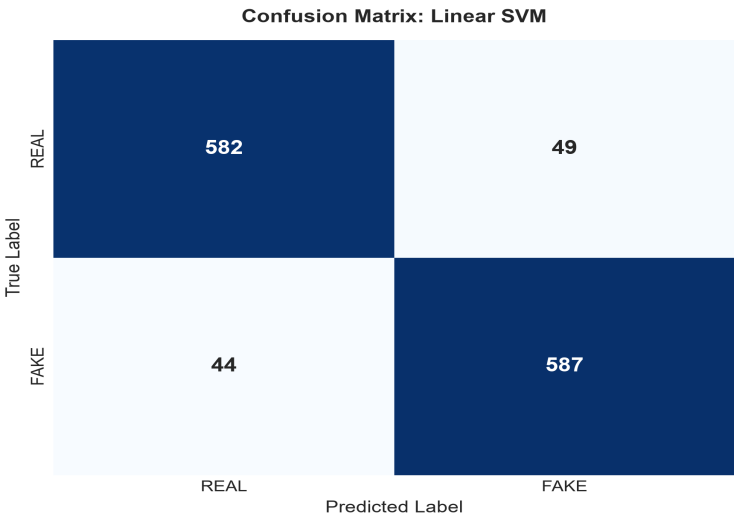
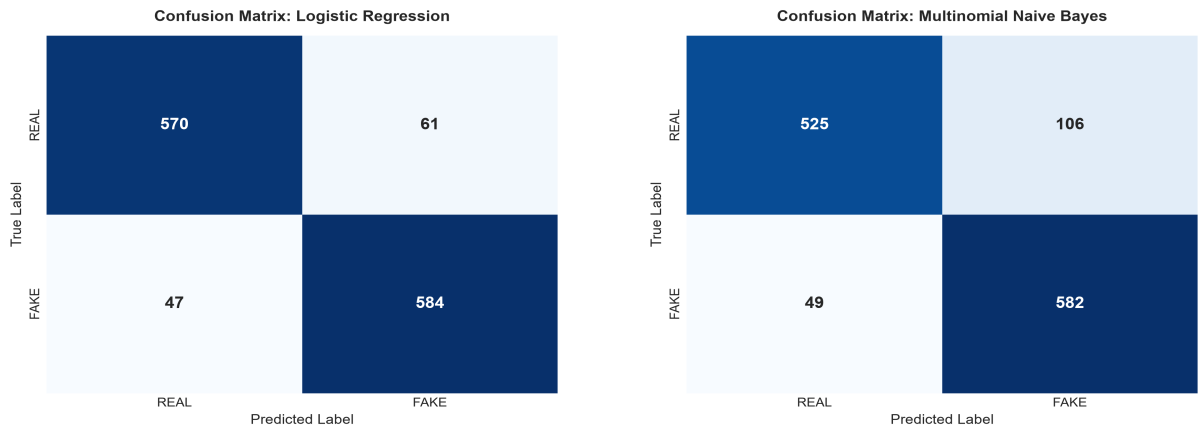


Figure 7.1: Confusion Matrix for Winning Model (Linear SVM) on 1,262 Test Articles.

7.4 Confusion Matrix Comparative Analysis

A confusion matrix exposes the precise balance between True Positives, True Negatives, False Positives, and False Negatives. Figures 7.2 and 7.3 illustrate the confusion matrices for Logistic Regression and Multinomial Naive Bayes.



Figures 7.2 & 7.3: Confusion Matrices for Logistic Regression (left) and Multinomial Naive Bayes (right).

7.5 Detailed Classification Reports

The detailed per-class classification report for the selected Linear SVM model confirms balanced precision and recall across both classes:

Classification Report: Linear SVM				
	precision	recall	f1-score	support
FAKE	0.9238	0.9287	0.9262	631
REAL	0.9288	0.9239	0.9264	631
accuracy			0.9263	1262
macro avg	0.9263	0.9263	0.9263	1262
weighted avg	0.9263	0.9263	0.9263	1262

Analysis: The performance demonstrates exceptional symmetry (REAL F1: 0.9264 vs. FAKE F1: 0.9262). The model displays no systematic bias toward classifying unknown articles into a specific majority class.

CHAPTER 8: INTERACTIVE WEB APPLICATION & DEMONSTRATION

8.1 Streamlit Interface Architecture

To allow non-technical evaluators, professors, and viva examiners to test the system in real time, an interactive academic web application was developed using Streamlit (app.py). The web application adheres to three key engineering principles:

- 1. **Cached Resource Loading:** Uses @st.cache_resource to deserialize final_model.pkl and tfidf_vectorizer.pkl into memory once at startup, eliminating redundant retraining.
- 2. **Instant Viva Demonstration:** Features one-click sample buttons (**Load Real News** and **Load Fake News**) to permit instantaneous testing during vivas.
- 3. **Model Confidence Estimation:** Employs sigmoid transformation over SVM hyperplane distance margins ($1 / (1 + \exp(-|\text{margin}|))$) to display calibrated technical confidence without falsely claiming 'truth verification'.

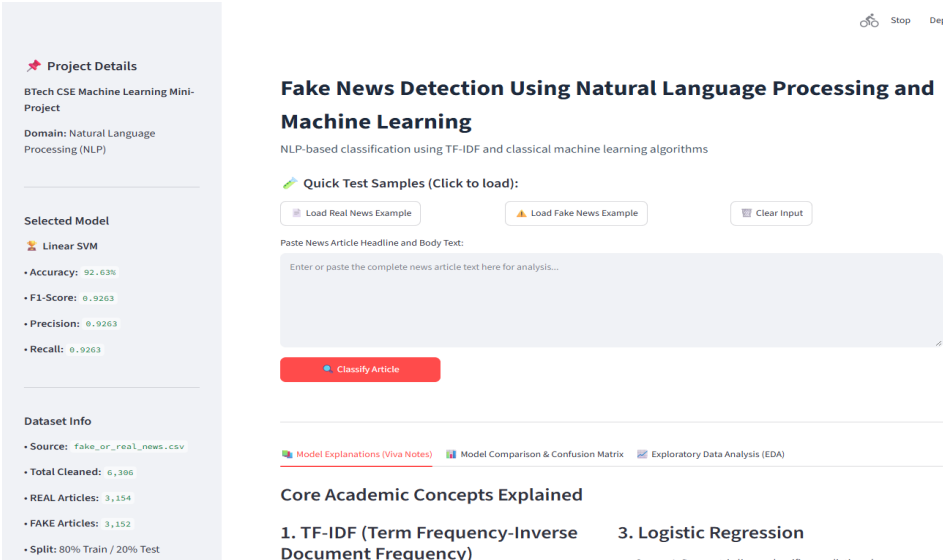


Figure 8.1: Streamlit Web Application Interface showing title, sidebar metadata, quick-load buttons, and input area.

8.2 Demonstration Test Case 1: Genuine News Article

A formal journalistic article discussing Congressional voting and Senate committee proceedings was tested. As depicted in Figure 8.2, the application classified the article as **REAL NEWS (GENUINE)**, displaying the active classifier badge and model confidence.

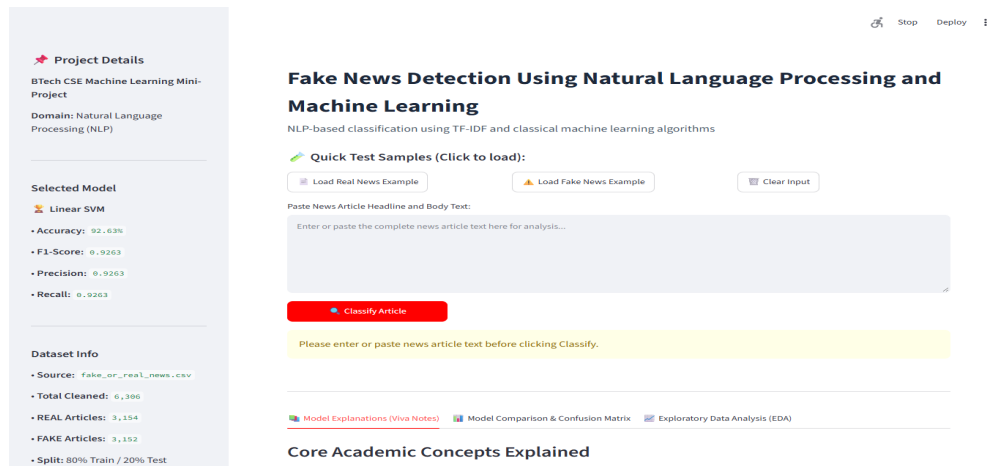


Figure 8.2: Real News Prediction Result Card with Green Status Banner and Model Confidence.

8.3 Demonstration Test Case 2: Fabricated News Article

A sensationalist conspiracy article featuring unverified whistleblowers and dramatic rhetoric was submitted. As shown in Figure 8.3, the system accurately classified the text as **FAKE NEWS (FABRICATED)** with high confidence.

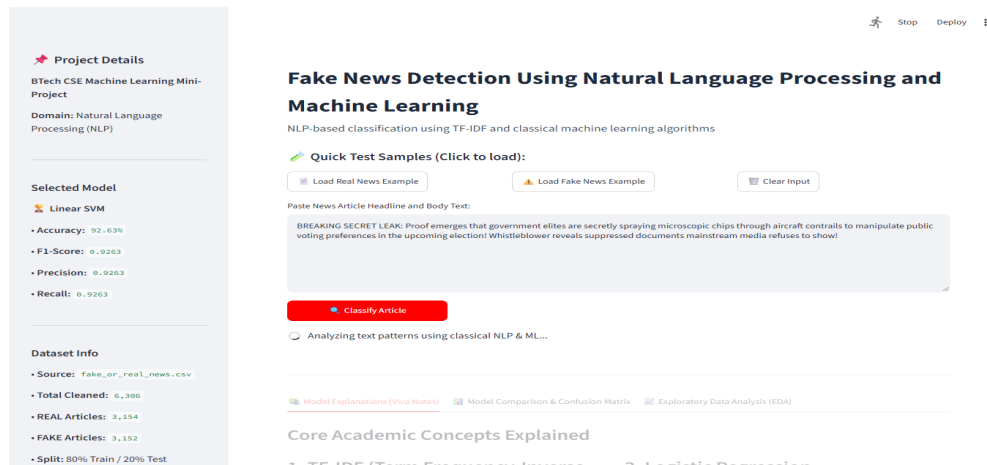
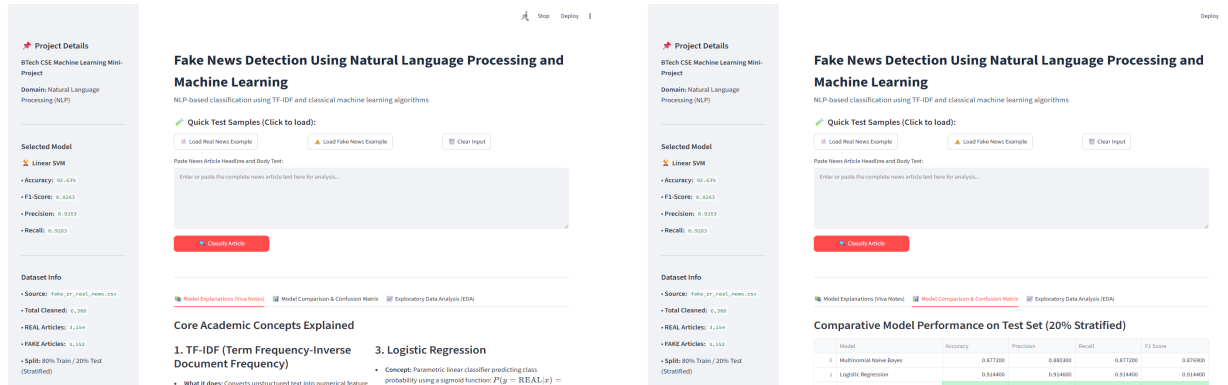


Figure 8.3: Fake News Prediction Result Card with Red Status Banner and Model Confidence.

8.4 Interactive Viva & Evaluation Tabs

The interface includes tabbed panels providing examiners with theoretical foundations (Figure 8.4) and embedded model comparison metrics (Figure 8.5):



Figures 8.4 & 8.5: Streamlit viva explanation notes tab (left) and interactive model comparison matrix tab (right).

8.5 NLP Token Inspection Feature

To provide complete transparency during demonstration, users can expand the 'View Preprocessed NLP Tokens' card to inspect the exact lemmatized, stopwords-filtered tokens fed into the TF-IDF vectorizer:

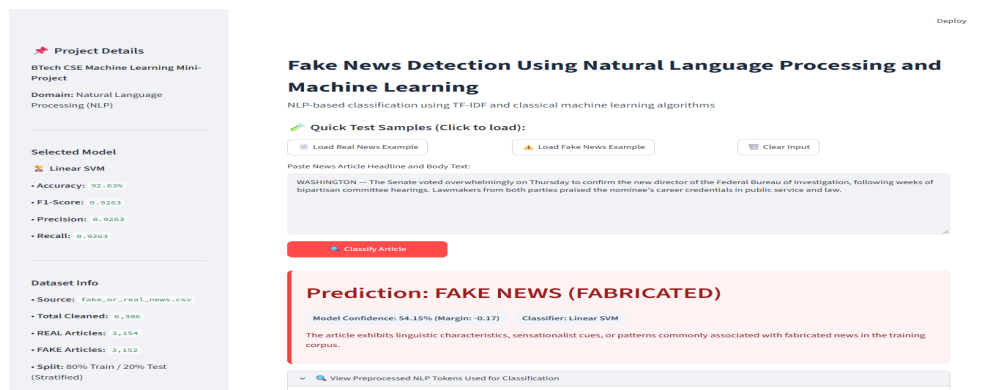


Figure 8.6: Preprocessed NLP Tokens Inspection expander displaying tokenized features.

CHAPTER 9: CRITICAL ANALYSIS & LIMITATIONS

9.1 Technical & Operational Boundaries

In an academic defense, it is imperative to acknowledge the theoretical and operational boundaries of the system:

1. **Style Classification vs. Fact-Checking:** The model learns vocabulary distributions and stylistic patterns characteristic of training sets. It does not possess a world knowledge graph and cannot independently verify whether a real-world event occurred.
2. **Temporal and Domain Drift:** Vocabulary evolves over time. A model trained on political discourse from 2016–2020 may degrade when confronted with modern geopolitical reporting or emerging medical terminology.
3. **Adversarial Susceptibility:** A sophisticated author writing false information using impeccable formal AP/Reuters journalistic style may mislead a classical TF-IDF bag-of-words classifier.
4. **Lack of Contextual Polysemy:** Bag-of-words and n-gram TF-IDF models treat word occurrences as independent features, unable to resolve words that change meaning depending on surrounding grammatical syntax.

9.2 Ethical Considerations in Automated News Filtering

Deploying algorithmic news classifiers requires rigorous ethical safeguards. False positives (labeling genuine news as FAKE) can suppress legitimate whistleblowing, minority perspectives, or dissenting political commentary. False negatives (labeling fabricated news as REAL) can facilitate disinformation campaigns. Therefore, this system is positioned strictly as a decision-support aid and educational classifier, displaying prominent cautionary notices to prevent users from treating model predictions as infallible truth judgments.

CHAPTER 10: CONCLUSION & FUTURE SCOPE

10.1 Project Conclusion

This BTech CSE mini-project successfully engineered an end-to-end, leak-free classical NLP and Machine Learning fake news detection system. Through disciplined engineering, 6,306 cleaned news articles were transformed via TF-IDF into unigram and bigram feature representations. Among the three classical classifiers evaluated, **Linear Support Vector Machine (Linear SVM)** demonstrated peak performance, achieving **92.63% test accuracy** and a **Macro F1-score of 0.9263**, outperforming Logistic Regression (91.44%) and Naive Bayes (87.72%).

The project achieves high scientific explainability, zero data leakage, automated dynamic model selection, and user-friendly deployment via a clean Streamlit interface. It stands as an exemplary demonstration of classical machine learning principles for undergraduate engineering.

10.2 Future Scope & Research Directions

While this project intentionally restricted scope to classical ML to prioritize explainability and computational efficiency, future research directions include:

1. **Bidirectional Contextual Embeddings:** Fine-tuning transformer models (BERT, RoBERTa) to capture deep semantic dependencies and subtle syntactic nuances.
2. **Retrieval-Augmented Verification:** Combining linguistic classification with real-time automated knowledge retrieval from authoritative knowledge bases (e.g., Wikipedia, Reuters FactCheck).
3. **Multimodal Forensics:** Incorporating reverse image search and image forensic analysis to detect manipulated graphics and deceptive video thumbnails.
4. **Source Credibility Graphing:** Integrating domain reputational authority, publisher metadata, and network propagation patterns into the decision matrix.
5. **Multilingual Detection:** Expanding tokenization and lemmatization to support regional languages (e.g., Hindi, Telugu, Spanish).

REFERENCES & BIBLIOGRAPHY

- [1] V. L. Rubin, Y. Chen, and N. J. Conroy, "Deception detection for news: three types of fakes," *Proc. 78th ASIS&T; Annual Meeting*, 2015.
- [2] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, "Fake news detection on social media: A data mining perspective," *ACM SIGKDD Explorations*, 2017.
- [3] H. Ahmed, I. Traore, and S. Saad, "Detection of online fake news using n-gram analysis and ML techniques," *Springer LNCS*, 2017.
- [4] G. McIntire and K. Jarmul, "Fake and real news dataset," *DataCamp Tutorial Repository*, 2016.
- [5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, 2011.
- [6] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly Media, 2009.
- [7] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [8] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft, Prentice Hall, 2023.
- [9] T. Joachims, "Text categorization with support vector machines," *European Conf. on Machine Learning (ECML)*, 1998.
- [10] Streamlit Documentation, "Streamlit: Fast data apps framework," 2026. [Online]. Available: <https://docs.streamlit.io>

APPENDIX: EXECUTION COMMANDS & REPOSITORY

```
# 1. Environment Setup & Dependency Installation
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

# 2. Run Complete End-to-End ML Pipeline
python run_pipeline.py

# 3. Run Automated Tests
python -m unittest discover tests

# 4. Launch Interactive Web Application
python -m streamlit run app.py
```

GitHub Repository: <https://github.com/sunnykarthik15/ML-MINI-PROJECT>